

RAG Pipeline

Architecture & Implementation Guide

drilldown-cap-int · feat/RAG/sid
Generated June 23, 2026

A technical deep-dive into the Retrieval-Augmented Generation ingestion and retrieval pipeline powering the drill-down explainer feature.

Embedding Model all-MiniLM-L6-v2	Vector Dims 384	Chunk Size 400 words
Overlap 80 words	Vector DB Qdrant (local)	Vision LLM Qwen 2.5VL 7B

1. Overview

The RAG (Retrieval-Augmented Generation) pipeline in *drilldown-cap-int* enriches drill-down component explanations with grounded, citation-backed context extracted from user-uploaded PDF documents. Instead of relying solely on a vision language model's parametric knowledge, the system first searches an indexed knowledge base and injects the most relevant passage into the generation prompt — providing verifiable, document-specific answers.

The pipeline has two primary phases: **Ingestion** (offline, triggered on PDF upload) and **Retrieval** (online, triggered on each drill-down query). Both phases share the same embedding model to ensure query-document vector space alignment.

Stage	Trigger	Output
Ingestion	POST /api/kb/upload	Qdrant collection + registry entry
Retrieval	Drill-down request with kb_id	Top-k chunks + citations injected into prompt

2. Ingestion Pipeline

When a user uploads a PDF via the knowledge base UI, the following pipeline runs synchronously (inside a thread executor so the async FastAPI event loop is not blocked). The six phases below transform raw bytes into indexed, searchable vector embeddings.

Phase 1	Upload & ID Generation	controllers/kb.py
The FastAPI controller validates the uploaded file is a PDF, reads the raw bytes, and derives a content-based identifier using SHA-256. The first 16 hex characters become the kb_id, making re-uploads of the same file fully idempotent.		
Entry point	handle_upload_kb()	
ID scheme	sha256(pdf_bytes)[:16] → e.g. a942cbba4b145e1a	
Validation	MIME check – 400 returned if not application/pdf	

A global `DocumentConverter` singleton (from the `docling` library) parses the PDF into a typed element stream. Each element is classified into one of five types. The last seen `SectionHeaderItem` is tracked and prepended as context to subsequent text and list items, preserving the document's logical hierarchy.

Library	<code>docling · DocumentConverter</code> (lazy singleton)
Elements	<code>SectionHeaderItem</code> , <code>TextItem</code> , <code>ListItem</code> , <code>TableItem</code> , <code>PictureItem</code>
Context	<code>[section header]</code> text – prepended to text/list items
Tables	<code>export_to_markdown()</code> → single Markdown string

Each `PictureItem` is exported to PNG bytes and sent to a locally running Ollama instance with the Qwen 2.5VL 7B vision model. The function detects whether the figure is a flowchart (via keyword matching on the caption) and tailors the prompt accordingly. Flowcharts receive step-by-step numbered extraction; other diagrams receive component, label, and connection descriptions. If the VLM is unavailable, the raw caption is used as a graceful fallback.

VLM	<code>qwen2.5vl:7b</code> via Ollama (http://localhost:11434)
Flowchart	Keyword detect: <code>'flow'</code> , <code>'procedure'</code> , <code>'cycle'</code> , <code>'diagram'</code> , ...
Output	Step-by-step (IF/THEN/ELSE) or component description
Fallback	Caption text if VLM call fails or Ollama is not running

Phase
4

Chunking

ingestion.py · `_chunk_text()`

Parsed elements are chunked using a sliding word-window approach. Text and list items use the standard window with overlap to preserve cross-chunk context. Tables and figures are kept as single atomic chunks unless they exceed double the chunk size, in which case they are split at word boundaries. Every chunk retains its source page number and content type for downstream citation.

Chunk size	<code>_CHUNK_SIZE = 400 words</code>
Overlap	<code>_CHUNK_OVERLAP = 80 words</code>
Tables	Single chunk; split only if > 800 words
Figures	Single chunk; split only if > 800 words
Metadata	<code>page_num</code> , <code>content_type</code> , <code>chunk_index</code> preserved per chunk

Phase
5

Embedding

embedder.py · `get_embedder()`

All chunk texts are batch-encoded by the all-MiniLM-L6-v2 sentence-transformer model. The model is loaded once at startup and cached for the process lifetime. The same model is used at retrieval time, guaranteeing that query vectors and document vectors live in the same semantic space.

Model	<code>sentence-transformers/all-MiniLM-L6-v2</code>
Dimensions	<code>384 (float32)</code>
Distance	<code>Cosine similarity</code>
Batch size	<code>32 samples per encode() call</code>
Lifecycle	<code>Lazy singleton – loaded once, reused for queries</code>

Embedded points are upserted into a Qdrant collection named `kb_{kb_id}`. The collection is dropped and recreated on each upload, making the operation fully idempotent. Points are written in batches of 100. After storage completes, `register_kb()` appends a metadata record to the JSON registry.

Collection	<code>kb_{sha256[:16]}</code> – one isolated collection per KB
Point ID	<code>uint32 from md5('{kb_id}_{chunk_idx}')[:8]</code>
Payload	<code>text, page_num, source_name, content_type, chunk_index</code>
Batch size	100 points per upsert call
Registry	<code>data/kb/registry.json</code> – <code>id, name, page_count, created_at</code>
DB path	<code>data/kb/qdrant/</code> (local file-based Qdrant)

3. Retrieval Pipeline

Retrieval is triggered at query time when a drill-down request includes a `kb_id`. The system encodes the free-text analysis query, searches the corresponding Qdrant collection, and injects the best-matching chunk into the LLM generation prompt as grounded context.

1	Query arrives Drill-down request contains <code>kb_id</code> , triggering <code>_enrich_with_kb()</code>	<code>drill_context_resolver.py</code>
2	Embed query The same <code>all-MiniLM-L6-v2</code> model encodes the component analysis text → <code>float32[384]</code>	<code>retriever.py</code> · <code>search_kb()</code>
3	Vector search Qdrant cosine search over <code>kb_{kb_id}</code> with <code>top_k=4</code> , returning payloads + scores	<code>QdrantClient.query_points()</code>
4	Inject context Top hit's text is added to the LLM prompt; 2nd–4th hits stored as <code>kb_extra_hits</code>	<code>drill_context_resolver.py</code>
5	Citation attached Result includes <code>kb_citation</code> ('filename, page N'), <code>kb_score</code> , <code>kb_mode=True</code>	<code>DrillResult</code> payload

3.1 Retrieval Result Schema

```
{ "text": str, # Chunk content "page_num": int, # Source PDF page "source_name": str, # Original filename "score": float, # Cosine similarity (0-1) "content_type": str # "text" | "table" | "figure" }
```

3.2 Enriched Drill-Down Fields

When KB retrieval succeeds, the following fields are added to the drill-down result and surfaced in the metadata panel:

Field	Type	Description
<code>kb_description</code>	<code>str</code>	Top chunk text injected into prompt

kb_citation	str	"filename, page N" — shown to user
kb_score	float	Cosine similarity of top hit
kb_extra_hits	list	2nd–4th results for optional display
kb_mode	bool	True when KB enrichment active

4. Data Model

4.1 Qdrant Point Structure

```
PointStruct( id = uint32 # md5(f'{kb_id}_{chunk_index}')[ :8] vector = float32[384]
# cosine distance space payload = { "text": str, # chunk text (400-word window)
"page_num": int, # 1-indexed page from Docling "source_name": str, # original PDF
filename "chunk_index": int, # position in document order "content_type": str #
"text" | "table" | "figure" } )
```

4.2 Registry Record (registry.json)

```
{ "id": "a942cbb4b145e1a", // sha256(bytes)[:16] "name": "equipment_manual.pdf",
// original filename "page_count": 312, // chunk count (not pages) "created_at":
"2026-06-23T14:00:00Z" // ISO 8601 UTC }
```

4.3 Collection Naming & Isolation

Each uploaded PDF gets its own isolated Qdrant collection named `kb_{sha256[:16]}`. This means queries never cross-contaminate between knowledge bases, and deleting a KB is a simple collection drop — no filtering required. The tradeoff is that multi-KB cross-search would require aggregating results across collections manually.

5. Configuration Reference

All settings are defined in `backend/shared/config.py` as a Pydantic *Settings* class and can be overridden via environment variables.

Env Var	Default	Purpose
KB_DIR	data/kb	Registry JSON location
KB_QDRANT_PATH	data/kb/qdrant	Local Qdrant database root
OLLAMA_BASE	http://localhost:11434	Ollama base URL for VLM calls
VISION_MODEL	qwen2.5vl:7b	Figure description model
QDRANT_PATH	data/qdrant_storage	Separate ecommerce Qdrant DB
COLLECTION_NAME	fashion_products	Ecommerce vector collection

6. API Endpoints

Method	Path	Handler	Description
POST	/api/kb/upload	handle_upload_kb()	Upload PDF, trigger ingestion
GET	/api/kb/list	handle_list_kbs()	List all registered KBs
DELETE	/api/kb/{kb_id}	handle_delete_kb()	Delete KB + Qdrant collection

7. Key Design Decisions

Content-based IDs

`sha256(pdf_bytes)[:16]` means the same file always produces the same `kb_id`. Re-uploading is idempotent — the Qdrant collection is dropped and rebuilt, avoiding stale index state without needing version tracking.

Per-KB Qdrant collections

Isolating each KB in its own collection (`kb_{id}`) avoids cross-contamination and makes deletion trivially safe. The tradeoff is that cross-KB search requires manual aggregation across collections.

Shared embedding singleton

The all-MiniLM-L6-v2 model is loaded once per process and reused for both ingestion and queries. This guarantees vector space alignment and avoids paying the ~80MB model load cost on every request.

Local Qdrant (file-based)

Using Qdrant in local file mode (no separate server process) keeps deployment simple. The database lives at `data/kb/qdrant/`. If the collection count or volume grows significantly, migrating to a Qdrant server is straightforward.

Graceful VLM fallback

Figure description via Ollama is optional. If the VLM is unavailable or the call fails, the raw Docling caption is used instead. This means ingestion never hard-fails due to a missing local model.

Overlapping chunks

An 80-word overlap between consecutive 400-word chunks ensures that information near chunk boundaries is represented in both adjacent chunks, reducing the chance of a relevant fact being split across a boundary and missed by retrieval.

8. File Map

Path	Purpose
backend/services/knowledge_base/ingestion.py	Core ingestion: PDF parsing, chunking, embedding, Qdrant upsert
backend/services/knowledge_base/embedder.py	Singleton embedder: all-MiniLM-L6-v2 loader + encode()
backend/services/knowledge_base/retriever.py	Vector search: search_kb() → ranked top-k results
backend/services/knowledge_base/store.py	JSON registry: register_kb(), list_kbs(), remove_kb()
backend/app/routes/kb.py	FastAPI router: /api/kb/upload, /list, /{kb_id}
backend/app/controllers/kb.py	Request handlers: file validation, thread-executor dispatch
backend/services/explainer/drill_context_resolver.py	KB enrichment: _enrich_with_kb() injects context into drill prompt
backend/shared/config.py	Settings: KB_DIR, KB_QDRANT_PATH, OLLAMA_BASE, VISION_MODEL
data/kb/registry.json	Runtime: JSON array of registered KB metadata records
data/kb/qdrant/	Runtime: Local Qdrant database directory (one collection per KB)
test_ingestion.py	Smoke test: end-to-end ingestion + query validation